

```
1 package Expense;
2
3 /**
4  * Write a description of class JDBCExample here.
5  *
6  * @author (your name)
7  * @version (a version number or a date)
8  */
9
10 // For MySQL connectivity
11 import java.sql.Connection;
12 import java.sql.DriverManager;
13 import java.sql.SQLException;
14 import java.sql.Statement;
15 import java.sql.ResultSet;
16
17 import java.util.*;
18
19 public class DBConnection {
20     static final String DB_URL = "jdbc:mysql://localhost:3306/Expenses"; //table
21     static final String DB_URL1 = "jdbc:mysql://localhost:3306/";
22     static final String USER = "root";
23     static final String PASS = "root123";
24
25     //Connecting to tge RDBMS with the DriverManager class involves calling the
26     method DriverManager.getConnection.
27
28     public Connection DBGetConn() {
29         // Open a connection
30         Connection conn = null;
31         //System.out.println("Connecting to Database...");
32         try
33         {
34             // Registering drivers
35             conn = DriverManager.getConnection(DB_URL, USER, PASS);
36             // System.out.println("Inside try block");
37             // System.out.println("Connected to database");
38         }
39         catch (SQLException e) {
40             e.printStackTrace();
41             //System.out.println("Inside catch block");
42         }
43         //System.out.println("Outside try/catch block");
44         return conn;
45     }
46
47     public static void DBCreate(String DBNAME)
48     {
49         try(Connection conn = DriverManager.getConnection(DB_URL1, USER, PASS);
50             Statement stmt = conn.createStatement();)
51         {
52             String sql = "CREATE DATABASE "+ DBNAME + " ";
53             stmt.executeUpdate(sql);
```

```
53         System.out.println("Database created successfully...");
54     }
55     catch (SQLException e) {
56         e.printStackTrace();
57         //System.out.println("Inside catch blockDB CREATE");
58     }
59     //System.out.println("Outside try/catch db create block");
60 }
61
62 public static void DBDel(String DBNAME)
63 {
64     try(Connection conn1 = DriverManager.getConnection(DB_URL1, USER, PASS);
65     Statement stmt = conn1.createStatement();)
66     {
67         String sql = "DROP DATABASE "+ DBNAME + " ";
68         stmt.executeUpdate(sql);
69         System.out.println("Database deleted successfully...");
70     }
71     catch (SQLException e) {
72         e.printStackTrace();
73         //System.out.println("Inside catch blockDB del");
74     }
75     //System.out.println("Outside try/catch db del block");
76 }
77
78
79 public static void DBList()
80 {
81     Statement stmt = null;
82     ResultSet resultset = null;
83
84     try {Connection conn1 = DriverManager.getConnection(DB_URL, USER, PASS);
85         stmt = conn1.createStatement();
86         resultset = stmt.executeQuery("SHOW DATABASES;");
87
88         while (resultset.next()) {
89             System.out.println(resultset.getString("Database"));
90         }
91     }
92     catch (SQLException ex){
93         // handle any errors
94         ex.printStackTrace();
95     }
96     finally {
97         // release resources
98         if (resultset != null) {
99             try {
100                 resultset.close();
101             } catch (SQLException sqlEx) { }
102             resultset = null;
103         }
104
105         if (stmt != null) {
106             try {
```

```
107         stmt.close();
108     } catch (SQLException sqlEx) { }
109     stmt = null;
110 }
111 }
112 }
113
114 public static void DBtableCreate()
115 {
116     try(Connection conn1 = DriverManager.getConnection(DB_URL, USER, PASS);
117     Statement stmt = conn1.createStatement();
118     ) {
119         String sql = "CREATE TABLE IF NOT EXISTS Expenses2022 " +
120             "(id INTEGER not NULL AUTO_INCREMENT, " +
121             " expense_name VARCHAR(255) NOT NULL, " +
122             " start_date DATE NOT NULL, " +
123             " expiry_date DATE NOT NULL, " +
124             " monthly_payment DECIMAL(10,2), " +
125             " yearly_payment DECIMAL(10,2), " +
126             " PRIMARY KEY ( id ))";
127
128         stmt.executeUpdate(sql);
129         System.out.println("Created table in the database...");
130     } catch (SQLException e) {
131         e.printStackTrace();
132     }
133     //System.out.println("Outside try/catch db del block");
134 }
135
136 public static void DBtableDel()
137 {
138     // Open a connection
139     try(Connection conn1 = DriverManager.getConnection(DB_URL, USER, PASS);
140     Statement stmt = conn1.createStatement();
141     ) {
142         String sql = "DROP TABLE expenses2022";
143         stmt.executeUpdate(sql);
144         System.out.println("Table deleted in the database...");
145     } catch (SQLException e) {
146         e.printStackTrace();
147     }
148 }
149
150
151 public static void DBtablelist()
152 {
153     Statement stmt = null;
154     ResultSet resultset = null;
155     try {Connection conn1 = DriverManager.getConnection(DB_URL, USER, PASS);
156         stmt = conn1.createStatement();
157         stmt.executeQuery("SHOW tables");
158
159         resultset = stmt.getResultSet();
160
```

```
161         while (resultset.next()) {
162             System.out.println(resultset.getString(1));
163         }
164     }
165     catch (SQLException ex){
166         // handle any errors
167         ex.printStackTrace();
168     }
169     finally {
170         // release resources
171         if (resultset != null) {
172             try {
173                 resultset.close();
174             } catch (SQLException sqlEx) { }
175             resultset = null;
176         }
177
178         if (stmt != null) {
179             try {
180                 stmt.close();
181             } catch (SQLException sqlEx) { }
182             stmt = null;
183         }
184     }
185 }
186 }
187 }
```

```
1 package Expense;
2
3
4 /**
5  * Write a description of class JDBCExample here.
6  *
7  * @author (your name)
8  * @version (a version number or a date)
9  */
10
11 //For JDBC
12 import java.sql.SQLException;
13 import java.sql.ResultSet;
14 import java.sql.ResultSetMetaData;
15 import java.sql.Statement;
16 import java.awt.*;
17
18 //Date formating
19 import java.text.SimpleDateFormat;
20 import java.util.Date;
21 import java.text.ParseException;
22
23 // to get user input,
24 import java.util.Scanner;
25
26 //i/o exception
27 import java.io.IOException;
28
29 //for JAVA GUI and Java GUI Table
30 import java.awt.Font;
31 import javax.swing.JFrame;
32 import javax.swing.JLabel;
33
34 import javax.swing.JScrollPane;
35 import javax.swing.JTable;
36 import javax.swing.ImageIcon;
37
38
39 public class ExpenseCalclater {
40     //Variables
41     static String QUERYDISPLAY = "SELECT id, Expense_Name, Yearly_Payment,
Monthly_Payment FROM Expenses2022";
42     //DB connection dbconn to be accessed
43     static DBConnection dbconn = new DBConnection();//why static
44
45     //The Statement objects allow you to execute basic SQL queries and retrieve
the results through the ResultSet class
46 //https://dev.mysql.com/doc/connector-j/8.0/en/connector-j-usagenotes-statements.h
tml
47     static Statement stmt = null;
48     static ResultSet rs = null;
49
50     //Methods
51
```

```
52     static void ClearResources()
53     {
54         if (rs != null) {
55             try {
56                 rs.close();
57             } catch (SQLException sqlEx) { } // ignore
58
59             rs = null;
60         }
61
62         if (stmt != null) {
63             try {
64                 stmt.close();
65             } catch (SQLException sqlEx) { } // ignore
66
67             stmt = null;
68         }
69     }
70 }
71
72 //this functions returns true if there are no records in the DB
73 static boolean IsRecordsEmpty() throws SQLException
74 {
75
76     //System.out.println("Static method IsRecordsEmpty");
77     //We could have simply used count(*) to get number of records but to print
column names I used the following code
78     //referencing to oracle site
79     String QUERYDISPLAY = "SELECT * FROM expenses2022";
80     try(
81     Statement stmt = dbconn.DBGetConn().createStatement();
82     //Executing the query
83     ResultSet rs = stmt.executeQuery(QUERYDISPLAY);)
84     {
85
86         //Retrieving the result
87         if (!rs.next()) {
88             System.out.print("No Expense Records in Database Table! Please add
the expenses first.");
89             // Get column names from table using ResultSetMetaData
90 //https://docs.oracle.com/javase/8/docs/api/java/sql/ResultSetMetaData.html
91             ResultSetMetaData rsmd = rs.getMetaData();
92             int columnCount = rsmd.getColumnCount();
93             System.out.println("Printing the columns in our Expense table :
");
94             // Get the column labels; column indices start from 1
95             for (int i=1; i<=columnCount; i++) {
96
97                 String columnName = rsmd.getColumnLabel(i);
98                 System.out.println(columnName);
99             }
100
101             return true;
102         }
```

```
103
104
105     }
106     catch (SQLException e) {
107         e.printStackTrace();
108         System.out.println("Expense table NOT found in the Database ");
109         return true;
110     }
111     return false;
112 }
113 /*
114     Others ways to declare an array String[][] myArray = new String[5][]; // OK
115     String[][] yourArray = new String[5][4];
116
117     Read more:
118     https://www.java67.com/2014/10/how-to-create-and-initialize-two-dimensional-array-
119     java-example.html#ixzz7dN0EQW44
120
121     */
122     static void DisplayTable() throws Exception
123     {
124         boolean Rempty = IsRecordsEmpty();
125         if (Rempty == true)
126         {
127             // System.out.print("No Expense Records in Database Table to display!
128             Please add the expenses!");
129         }
130
131         else
132         {
133             String QUERYROWS = "SELECT COUNT(*) FROM expenses2022";
134             stmt = dbconn.DBGetConn().createStatement();
135
136             //have an array of type object to accomodate different
137             datatypes from db instead of string
138
139             Object[][] Array_Expenses;
140             //Executing the query
141             int Array_row =0;
142
143             rs = stmt.executeQuery(QUERYROWS);
144             if (rs.next()) {
145                 /* this is important to get row numbers from mysql table
146                 to initialise our 2D arrays
147                 * we have 6 columns currentlyyyyyy ,refering the mysql
148                 create table
149
150                 */
151                 Array_row = rs.getInt(1);
152                 //System.out.println("Row Count " + rs.getInt(1) + " " +
153                 Array_row );
154
155                 Array_Expenses = new Object[Array_row][5];
156
157                 String QUERY = "SELECT * FROM expenses2022";
158                 stmt = dbconn.DBGetConn().createStatement();
159                 //Executing the query
160                 rs = stmt.executeQuery(QUERY);
```

```
150         int i=0;
151         while(rs.next()){
152             //Display values
153
154             //Converting Date object to String
155             Date sdate = rs.getDate("Start_Date");
156             String s_date = sdate.toString();
157             // System.out.println("Date: "+sdate.toString());
158
159             Date edate = rs.getDate("expiry_date");
160             String e_date = edate.toString();
161             //System.out.println("Date: "+edate.toString());
162             //String.valueOf(rs.getInt(1))
163
164             Array_Expenses[i][0] = rs.getString("expense_Name");
165             //System.out.print(Array_Expenses[i][0]+" ,");
166             Array_Expenses[i][1] = s_date;
167             //System.out.print(Array_Expenses[i][1]+" ,");
168             Array_Expenses[i][2] =
169             "£"+rs.getBigDecimal("Yearly_Payment");
170             //System.out.print(Array_Expenses[i][2]+" ,");
171             Array_Expenses[i][3] =
172             "£"+rs.getBigDecimal("Monthly_Payment");
173             //System.out.print(Array_Expenses[i][3]+" ,");
174             Array_Expenses[i][4] = e_date;
175             //System.out.print(Array_Expenses[i][4]);
176             i++;
177             //System.out.println(); //change line on console as
178             row comes to end in the matrix.
179             // System.out.println("Hi"+i);
180
181         }
182         try{
183             for (int d = 0; d < Array_Expenses.length; d++) {
184                 //this equals to the row in our matrix.
185                 for (int j = 0; j < Array_Expenses[d].length; j++)
186                 { //this equals to the column in each row.
187                     //System.out.print(Array_Expenses[d][j] + "
188                     ");
189                 }
190                 //System.out.println();
191             }
192         }
193         catch(Exception e){
194
195             System.out.print("Error : " +e);
196             return;
197         }
198
199         // With two dimensional array, set the columns of a table.
200         Additionally, we have set the rows using a one-dimensional array as shown below
201
202         JTable table = new JTable(Array_Expenses,new String[] ..
```



```
198         Font font = new Font("Verdana", Font.PLAIN, 12);
199         table.setFont(font);
200
201         JFrame frame = new JFrame();
202
203         frame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
204
205         //changing table icon image
206         //https://www.javatpoint.com/how-to-change-titlebar-icon-in-java-awt-swing
207         Image EMSIcon =
Toolkit.getDefaultToolkit().getImage("C:\\Users\\Gaj\\Downloads\\gb.png");
208         frame.setIconImage(EMSIcon);
209
210         //Title of frame
211
212         frame.setTitle("My Expenses");
213
214         //this will oveide the setSize to adjustable view table
215         //table.setPreferredScrollableViewportSize(table.getPreferredSize());
216
217         //frame.setSize(900,900);
218         frame.add(new JScrollPane(table));
219         frame.pack();
220
221         //Adjusting the size of window and setting its position
at top left hand corner (0,0)
222         Toolkit theKit = frame.getToolkit();
223         Dimension wndSize = theKit.getScreenSize();
224         frame.setBounds(0,0, //position
225         wndSize.width / 2, wndSize.height / 2); // Size
226
227         //frame.setExtendedState(JFrame.MAXIMIZED_FULL); to make
full screen of device
228         // Set's the window to be "always on top"
229         frame.setAlwaysOnTop( true );
230         //Displaying the window
231         frame.setVisible(true);
232     }
233
234
235 }
236 }
237
238 //Adds an expense to expense table in mysql DB
239 static void AddExpense()throws SQLException
240 {
241     try
242     {
243         System.out.print("Enter the Expense Name to be Added: ");
244         Scanner user_input = new Scanner(System.in);
245
246         String Ex_name;
247         double YPayment = 0;
```

```
248         double MPayment = 0;
249         //Expense name input
250         Ex_name = user_input.nextLine().toUpperCase();
251
252         //Client Request
253         String sql1="select * from expenses2022" +" WHERE expense_Name='"+
Ex_name +"' ";
254         Statement stmt1 = dbconn.DBGetConn().createStatement();
255         //System.out.print(sql1);
256
257         ResultSet rs1 = null;
258         //Executing the query
259         rs1 = stmt1.executeQuery(sql1);
260         if(rs1.next() == true)
261         {
262             System.out.print("\n");
263             System.out.print("The Expense Name already exists in the
database!"+"
\nIf want to edit this record Use option 3 of Main Menu.\nTaking you
back to the Main Menu now. ");
264             return;
265         }
266
267
268
269         System.out.println("You entered a Expense Name to be "+Ex_name);
270         //Check to see if only two characters are entered
271         if((Ex_name.length() <=2 ))
272         {
273             System.out.print("Please enter a Meaningful/Realistic
Expense Name!");
274             return;
275         }
276         //Check to see if duplicate data exist in the database.If so tell the
user.
277
278
279         //Start Date input
280         String Sst_date;
281         System.out.print("\n Enter the Start Date in the format dd/mm/yyyy:
");
282         Sst_date = user_input.next();
283         //date format yyyy/mm/dd as per client's request
284         //https://www.tutorialspoint.com/java/java_string_matches.htm
285
286         String pattern ="\\d{1,2}/\\d{1,2}/\\d{4}";
287         if (Sst_date.matches(pattern))
288         {
289             //System.out.println("Correct format");
290         }
291         else{
292             System.out.println("Wrong Date format/ Date, Please try again");
293             return;
294         }
295
296         //Monthly or Yearly Payment input
```

```
298
299     System.out.println("Press 1 to Enter Monthly Payment");
300     System.out.println("OR");
301     System.out.println("Press 2 to Enter Yearly Payment");
302
303
304     if(user_input.hasNextInt()) {
305
306         Confirm = user_input.nextInt();
307
308         while((Confirm !=1 ) && (Confirm !=2))
309         {
310
311             System.out.println("Invalid choice!!! Please make a Valid
Choice. \n\n");
312             System.out.println("Press 1 to Enter Monthly Payment");
313             System.out.println("OR");
314             System.out.println("Press 2 to Enter Yearly Payment");
315             Confirm = user_input.nextInt();
316         }
317
318         //Creating switch case branch based on choice made by user
319         switch(Confirm){
320             //If its monthly payment:
321             case 1:
322                 System.out.println("The choice is to Enter Monthly
Payment") ;
323                 System.out.print("Enter the Monthly Payment: ");
324                 MPayment = user_input.nextDouble();
325                 YPayment = MPayment * 12.0;
326                 break;
327
328             case 2:
329                 System.out.println("The choice is to Enter Yearly
Payment") ;
330                 System.out.print("Enter the Yearly Payment: ");
331                 YPayment = user_input.nextDouble();
332                 MPayment = YPayment/12.0;
333                 break;
334
335                 //default case to display the message invalid choice made
by the user
336             default:
337                 System.out.println("Invalid choice!!! Please make a Valid
Choice. \n\n");
338
339         }
340     }
341
342     else if(!user_input.hasNextInt())
343     {
344         System.out.println("Invalid choice!!! Please make a Valid
Choice.Taking you back to Main Menu \n");
345         return;
346
```

```
348         //End of Contract Date input
349         String uext_date;
350         System.out.print("Enter the Expiry Date in the format dd/mm/yyyy: ");
351         uext_date = user_input.next();
352
353         if (!uext_date.matches(pattern))
354             {   System.out.println("Wrong Date format/ Date, Please try
again");
355                 return;
356             }
357         //Creating the Statement object
358         //System.out.println("\ncalling db class.");
359         Statement stmt = dbconn.DBGetConn().createStatement();
360         //System.out.println("\n after connection.");
361         //inserting rows
362         String sql = "INSERT INTO Expenses2022
(Expense_Name,start_date,expiry_date,Monthly_Payment,Yearly_Payment)
VALUES('"+Ex_name+"',STR_TO_DATE('"+Sst_date+"','%d/%m/%Y'),STR_TO_DATE('"+uext_da
te+"','%d/%m/%Y'),'"+MPayment+"','"+YPayment+"')";
363         //System.out.println("Inserting the record into the table..." +
sql);
364         stmt.executeUpdate(sql);
365         System.out.println("\nThe above entered data is inserted into the
Database table.");
366         System.out.println("You can verify this using: MENU Option 4:
Display Expense Table.");
367         System.out.println("And Update if required using: MENU Option 3:
Update Expense.");
368         user_input.close();
369     }
370     catch (SQLException e) {
371         e.printStackTrace();
372
373         return;
374     }
375
376 }
377
378
379 static void Update_RemoveExpense(int Indicator) throws Exception
380 {
381     //System.out.println("Static method Remove/Update can be called without
creating objects");
382     boolean Rempty = IsRecordsEmpty();
383     if (Rempty == true)
384     {
385         System.out.print("No Expense Records in Database to Remove/Update!
Please add an Expense Record before Removing/Updating.");
386     }
387
388     else{
389         DisplayTable();
390         Scanner user_input = new Scanner(System.in);
391
392         String Ex_name;
```

```
394
395         System.out.print("\nCheck the table above and Enter the 'Expense Name'
which you want to Remove/Update: ");
396         Ex_name = user_input.nextLine().toUpperCase();
397         //System.out.print("1Expense name you entered"+Ex_name);
398
399         String sql="select * from expenses2022" +" WHERE expense_Name='"+
Ex_name +"' ";
400         stmt = dbconn.DBGetConn().createStatement();
401         //Executing the query
402         rs = stmt.executeQuery(sql);
403
404         if(rs.next() == false)
405         {
406             System.out.print("\nPlease enter a Valid Expense Name! ");
407             System.out.print("\nThe Expense Name does not exists in the
database! Try Again, taking you back the Main Menu now.");
408             return;
409         }
410
411         System.out.print("\nAre you sure you want to Remove/Update the Expense
entry for: "+ Ex_name );
412         System.out.print("\nKindly enter 1 for Confirmation or 2 to
Exit/Retry: ");
413
414         U_Confirm = user_input.nextInt();
415
416
417         System.out.print("\n Your Choice is :"+ U_Confirm +"\n");
418         if(U_Confirm == 1)
419         {
420             System.out.print("Choice Confirmed.");
421             String QUERYDELUP="";
422             if(Indicator == 0)
423             {
424                 QUERYDELUP = "DELETE FROM expenses2022" +" WHERE
expense_Name='"+ Ex_name +"' ";
425                 //System.out.println("SQL QUERY TO DEL RECORD"+ QUERYDELUP);
426                 System.out.println("Connecting to Database...");
427                 stmt = dbconn.DBGetConn().createStatement();
428                 //Executing the query
429                 stmt.executeUpdate(QUERYDELUP);
430                 DisplayTable();
431             }
432             else if (Indicator == 1)
433             {
434                 QUERYDELUP = "Select * from expenses2022 " + "WHERE
expense_Name='"+ Ex_name +"' ";
435                 System.out.println(" Connecting to Database...");
436                 stmt = dbconn.DBGetConn().createStatement();
437                 //Executing the query
438                 rs = stmt.executeQuery(QUERYDELUP);
439                 while(rs.next()){
440                     //Display values
```

```
441         System.out.println("    EXPENSE NAME: " +
rs.getString("expense_Name"));
442         System.out.println("    START DATE: " +
rs.getDate("start_date"));
443         System.out.println("    YEARLY PAYMENT: " +
rs.getDouble("Yearly_Payment"));
444         System.out.println("    MONTHLY PAYMENT: " +
rs.getDouble("Monthly_Payment"));
445         System.out.println("    EXPIRY DATE: " +
rs.getDate("expiry_date"));
446     }
447
448     System.out.println("Press 1 to Edit EXPENSE NAME");
449     System.out.println("Press 2 to Edit START DATE");
450     System.out.println("Press 3 to Edit YEARLY PAYMENT VALUE");
451     System.out.println("Press 4 to Edit MONTHLY PAYMENT VALUE");
452     System.out.println("Press 5 to Edit EXPIRY DATE");
453     System.out.println("Make your Choice");
454
455     int user_choice =0;
456     double YPay = 0;
457     double MPay = 0;
458     String ExpName="s";
459     String s_date="";
460     String e_date="";
461
462     //Check for invalid input choice
463     if(user_input.hasNextInt()) {
464         user_choice = user_input.nextInt();
465         //Creating switch case branch based on choice made by user
466         /*
467         at the end
468         next line
469         the int.
470         *
471         */
472         user_input.nextLine();
473         switch (user_choice) {
474
475             //First case add record to your expense table
476             case 1:
477                 System.out.print("Enter the new 'Expense
Name':");
478                 ExpName = user_input.nextLine().toUpperCase();
479
480                 QUERYDELUP = "UPDATE expenses2022" + " SET
expense_Name = '"+ExpName+"' WHERE expense_Name='"+ Ex_name +"' ";
481                 System.out.print("\n Connecting to Database...");
482                 //System.out.println("SQL QUERY TO Update RECORD"+
QUERYDELUP);
483                 stmt = dbconn.DBGetConn().createStatement();
484                 //Executing the query
```

```

485 stmt.executeUpdate(QUERYDELUP);
486 System.out.print("Check Table for Updation");
487 DisplayTable();
488 break;
489
490 //Second case remove a record in expense table
491 case 2:
492     System.out.println("To Edit START DATE " + "Please
enter new value in proper format 'dd/mm/yyyy':");
493     s_date = user_input.next();
494     //System.out.println(s_date);
495     try {
496         String pattern = "\\d{1,2}/\\d{1,2}/\\d{4}";
497         if (s_date.matches(pattern))
498         {
499             //System.out.println("Correct
format");
500         }
501         else{
502             System.out.println("Wrong Date format/
Date, Please try again");
503             return;
504         }
505
506         //System.out.println("The date you entered: "+
s_date);
507     }
508     catch (Exception e) {
509         System.out.print("Wrong date format, Try
again!");
510         return;
511     }
512     QUERYDELUP = "UPDATE expenses2022" + " SET
start_date = STR_TO_DATE('" + s_date + "', '%d/%m/%Y') WHERE expense_Name='" + Ex_name
+ "' ";
513
514     //System.out.println("SQL QUERY TO UPDATE RECORD"+
QUERYDELUP);
515     stmt = dbconn.DBGetConn().createStatement();
516     //Executing the query
517     stmt.executeUpdate(QUERYDELUP);
518     System.out.print("Check Table for Updation");
519     DisplayTable();
520     break;
521
522     //Third case Update record in your expense table
523     case 3:
524         System.out.println("To Edit YEARLY PAYMENT VALUE,
" + "Please enter new value :");
525         YPay = user_input.nextDouble();
526         MPay = (YPay/12);
527         QUERYDELUP = "UPDATE expenses2022" + " SET
Yearly_Payment = '" + YPay + "', Monthly_Payment = '" + MPay + "' WHERE expense_Name='" +
Ex_name + "' ";
528
529         //System.out.println("SQL QUERY TO EDIT RECORD"+

```

```
531 stmt.executeUpdate(QUERYDELUP);
532 System.out.print("Check Table for Updation");
533 DisplayTable();
534 break;
535
536 //Fourth case DisplayTable()
537 case 4:
538     System.out.println("To Edit MONTHLY PAYMENT VALUE,
539 " + "Please enter new value: ");
540
541     MPay = user_input.nextDouble();
542     YPay = MPay * 12;
543     QUERYDELUP = "UPDATE expenses2022" + " SET
544 Yearly_Payment = '" + YPay + "', Monthly_Payment = '" + MPay + "' WHERE expense_Name='" +
545 Ex_name + "' ";
546
547     //System.out.println("SQL QUERY TO EDIT RECORD"+
548 QUERYDELUP);
549
550     stmt = dbconn.DBGetConn().createStatement();
551     //Executing the query
552     stmt.executeUpdate(QUERYDELUP);
553     System.out.print("Check Table for Updation");
554     DisplayTable();
555     break;
556
557 case 5:
558     System.out.println("To Edit Contract End Date, "
559 + "Please enter new value in proper format 'dd/mm/yyyy': ");
560     e_date = user_input.next();
561     //System.out.println(e_date);
562
563     try {
564         String pattern = "\\d{1,2}/\\d{1,2}/\\d{4}";
565         if (e_date.matches(pattern))
566         {
567             //System.out.println("Correct format");
568         }
569         else{
570             System.out.println("Wrong Date format/ Date,
571 Please try again");
572             return;
573         }
574         //System.out.println(e_date);
575     }
576     catch (Exception e) {
577         System.out.print("Wrong date format");
578         return;
579     }
580
581     QUERYDELUP = "UPDATE expenses2022" + " SET
582 expiry_date = STR_TO_DATE('" + e_date + "', '%d/%m/%Y') WHERE expense_Name='" + Ex_name
583 + "' ";
584
585     //System.out.println("SQL QUERY TO EDIT RECORD"+
586 QUERYDELUP);
587
588     stmt = dbconn.DBGetConn().createStatement();
589     ...
```



```
578         stmt.executeUpdate(QUERYDELUP);
579         System.out.print("Check Table for Updation");
580         DisplayTable();
581         break;
582
583         //default case to display the message invalid
584 choice made by the user
585         default:
586             System.out.println("Invalid choice!!! Please make
587 a valid choice. Back to main menu>");
588
589             } // case
590         } //if
591
592         else {
593             System.out.println("Invalid choice!!! Make a valid integer
594 Choice. Back to main menu.");
595         }
596
597         } //else if (1)
598
599     }
600     else if (U_Confirm == 2)
601     {
602         System.out.print("Going back to Main Menu");
603     }
604
605     else
606     {
607         System.out.print("Invalid entry! Kindly enter 1 for confirmation
608 or 2 to Exit/Retry");
609     }
610
611     }
612
613
614
615
616     static void CalculateExpense()
617     {
618         //difference between print and println
619         //System.out.println("Static method Calculate and Display Yearly or
620 monthly can be called without creating objects");
621         String QUERYCal = "SELECT SUM(Yearly_Payment),SUM(Monthly_Payment) FROM
622 expenses2022";
623         try{
624             stmt = dbconn.DBGetConn().createStatement();
625             //Executing the query
626             rs = stmt.executeQuery(QUERYCal);
627             while(rs.next()){
628                 //Display values
```

```
627         System.out.print("\n");
628     System.out.println("*****");
629         System.out.println("                Total Yearly Expense: "
+ "£"+rs.getBigDecimal(1));
630         System.out.println("                Totaly Monthly Expense: " + "£"+
rs.getBigDecimal(2));
631     System.out.println("*****");
632         System.out.print("\n");
633     }
634
635 }
636
637     catch (SQLException e) {
638         e.printStackTrace();
639         System.out.println("SQLException: " + e.getMessage());
640         System.out.println("SQLState: " + e.getSQLState());
641         System.out.println("VendorError: " + e.getErrorCode());
642         return;
643     }
644 }
645
646     //Added after client request
647 static void DisplayAnExpense() throws Exception
648 {
649
650     //System.out.println("Static DisplayAnExpense Method, can be called without
creating objects");
651     //Checking if records are empty
652     boolean Rempty = IsRecordsEmpty();
653     if (Rempty == true)
654     {
655         System.out.print("No Expense Records in Database to
Display!");
656     }
657
658     else{
659         // DisplayTable();
660         Scanner user_input = new Scanner(System.in);
661
662         String Ex_name;
663         int U_Confirm;
664
665         System.out.print("\nEnter the 'Expense Name' which you want
to be displayed: ");
666         //Client Request
667         Ex_name = user_input.nextLine().toUpperCase();
668         String sql="select * from expenses2022" +" WHERE
expense_Name='"+ Ex_name +"' ";
669         stmt = dbconn.DBGetConn().createStatement();
670         //System.out.print(sql);
671         ResultSet rs2 = null;
672         //Executing the query
673
```

```
675         if(rs2.next() == false)
676         {
677
678             System.out.print("\nPlease enter a Valid Expense Name!
679 ");
680             System.out.print("The Expense Name does not exists in
681 the database!\n Try Again, taking you back the Main Menu now. ");
682             return;
683         }
684
685         try{
686             rs = stmt.executeQuery(sql);
687             //System.out.print("inside try block");
688             int size = 0;
689             while(rs.next()){
690                 //Get Row numbers fr above query
691                 size++;
692             }
693             // System.out.print(" ROWS: " + size);
694             Object [][] Array_Expenses;
695             Array_Expenses = new Object[size][5];
696             rs = stmt.executeQuery(sql);
697             int i=0;
698             //lets populate a 2 D array to pass to Jtable
699             while(rs.next()){
700                 //Display values
701                 Array_Expenses[i][0] =
702 rs.getString("expense_Name");
703                 System.out.print(Array_Expenses[i][0]+"
704 ,");
705                 Array_Expenses[i][1] =
706 rs.getDate("start_date");
707                 System.out.print(Array_Expenses[i][1]+"
708 ,");
709                 Array_Expenses[i][2] =
710 "£"+rs.getBigDecimal("Yearly_Payment");
711                 System.out.print(Array_Expenses[i][2]+"
712 ,");
713                 Array_Expenses[i][3] =
714 "£"+rs.getBigDecimal("Monthly_Payment");
715                 System.out.print(Array_Expenses[i][3]+"
716 ,");
717                 Array_Expenses[i][4] =
718 rs.getDate("expiry_date");
719                 System.out.print(Array_Expenses[i][4]);
720                 System.out.print("\n");
721                 System.out.println();
722                 i++;
723             }
```

```
721         table.setFont(font);
722         table.setBackground(Color.blue);
723         //table.setBackground(new Color(46, 46,
46)); //black
724         table.setForeground(Color.white);
725         table.setRowHeight(30);
726
727         JFrame frame = new JFrame();
728         frame.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
729
730
731         Image EMSIcon =
Toolkit.getDefaultToolkit().getImage("C:\\Users\\Gaj\\Downloads\\gb.png");
732         frame.setIconImage(EMSIcon);
733
734         frame.setTitle("My Expense");
735
736         frame.setPreferredSize(new Dimension(600,
100));
737
738         //Jscroll pane is needed to display column
headers in this case.
739         //Table is child of scrollPane.
740
741         frame.add(new JScrollPane(table));
742
743         frame.pack();
744         // Set's the window to be "always on
top"
745         frame.setAlwaysOnTop( true );
746         //Display the window.
747         frame.setVisible(true);
748
749
750     }
751
752     catch (SQLException e) {
753         System.out.print("Couldn't print the Expense Record
:" + e);
754     }
755     user_input.close();
756 }
757
758
759 }
760
761 }
```

```
1 package Expense;
2
3 /**
4  * Write a description of class EmsLogin here.
5  * This class starts a session of EMS through a user login
6  * Passwords are given to the users of system
7  * A JFrame Window GUI is created for authentication purposes
8  * On successful authentication the EMS menu is presented to the user
9  * @version ( version number:V1 or a 11/08/2022)
10 **/
11
12 //Importing the Java GUI packages
13 import javax.swing.*;
14 import java.awt.*; //abstract window toolkit - later replaced by Swing
15 import java.awt.event.*;
16
17 //User input and Exception classes
18 import java.util.Scanner;
19 import java.io.IOException;
20
21 /*
22  * An interface is an abstract "class" that is used to group related methods with
23  "empty" bodies:
24  * To access the interface methods, the interface must be "implemented" (like
25  being inherited) by another class
26  with the implements keyword (instead of extends).
27
28  * The body of the interface method is provided by the "implement" class:
29  Note: Below we implement an ActionListener that listens to an event. The
30  ActionListener class can be found in the java.awt.event package.
31  It has only one method actionPerformed().
32  Here it keeps listening to any change on the button - if there is a click, and if
33  so performs an action (display the EMS menu).
34
35  (Here, the action is attached to a button. The action event is triggered on the
36  button click.)
37  */
38
39 public class EMSLogin extends JFrame implements ActionListener {
40     // Class Variables
41     private static JLabel password1, label;
42     private static JTextField username;
43     private static JButton button;
44     private static JPasswordField Password;
45     //This variable is needed to handle the multithread situation for control
46     flow.
47     private static int loginsuccess = 0;
48
49     public EMSLogin() {
50         /*Adding JPanel to the window
51         JPanel class covers the entire space or a window where we attach the
52         component to visualize it along with other panels.
53         Instantiates a JPanel class using the variable name: panel. Then, we set
54         the layout to be a null value,
55         meaning: the layout should take the entire width and height of the screen.
```

```
49
50     // creating a JPanel class
51     JPanel panel = new JPanel(null);
52
53     //Set JFrame parameters
54     //Setting the JFrame title name that will appear at the top of the frame
55     setTitle("Expense Management System");
56
57     //Positioning the frame on the window - 100 pixels from the left, 100
pixels from the top of the screen
58     setLocation(new Point(100, 100));
59
60     //We add the panel to the frame
61     add(panel);
62
63     //Sets the size of the entire frame - 400 pixels wide (width), 200 pixels
long (height)
64     setSize(new Dimension(400, 200));
65
66     //This will end and close the process whenever the user exits the window
67     setDefaultCloseOperation(HIDE_ON_CLOSE);
68
69     Image EMSIcon =
Toolkit.getDefaultToolkit().getImage("C:\\Users\\Gaj\\Downloads\\ganapa.jpg");
70     setIconImage(EMSIcon);
71     //Adding a window listen to frame.
72     addWindowListener(new WindowAdapter() {
73         public void windowClosing(WindowEvent we) {
74             if(loginsuccess == 0 || loginsuccess == 2){
75                 //System.out.println("Debug: I am here inside if close\n"
+loginsuccess);
76                 System.exit(0);
77             }
78             // System.out.println("Debug: I am here outside if
close\n"+loginsuccess);
79         }
80     });
81
82     //Username label constructor
83     label = new JLabel("Username");
84     label.setBounds(100, 8, 70, 20);
85     panel.add(label);
86
87     //Username TextField constructor
88     username = new JTextField();
89     username.setBounds(100, 27, 193, 28);
90     panel.add(username);
91
92     //Password Label constructor
93     password1 = new JLabel("Password");
94     password1.setBounds(100, 55, 70, 20);
95     panel.add(password1);
96
97     //Password TextField
98     Password = new JPasswordField();
```

```
99 Password.setBounds(100, 75, 193, 28);
100 panel.add>Password);
101
102 //Button constructor .Add the ActionListener to JButton object to
103 //listen to button clicks and handle the event in
104 // method :actionPerformed(ActionEvent e)
105
106 button = new JButton("Login");
107 button.setBounds(100, 110, 90, 25);
108 button.setForeground(Color.WHITE);
109 button.setBackground(Color.BLACK);
110 button.addActionListener(this);
111
112 panel.add(button);
113 setAlwaysOnTop(true);
114 setVisible(true);
115 // System.out.println("Debug: I am here in the EMS login\n\n");
116 }
117
118 private void DisplayUmenu() throws Exception
119 {
120     //Declaring variables
121     int choice;
122
123     //Creating an infinite while loop so control is returned back to the EMS
main menu
124     //after the appropriate function is called based on a choice made by the
user
125     while(true) {
126         System.out.println("\n-----");
127             System.out.println("----- MAIN MENU
-----");
128         System.out.println("-----");
129
130         //Creating menu
131         System.out.println("\n- Press 1 to Add Expense
-");
132         System.out.println("- Press 2 to Remove Expense
-");
133         System.out.println("- Press 3 to Update Expense
-");
134         System.out.println("- Press 4 to Display Expense Table
-");
135         System.out.println("- Press 5 to Display Yearly/Monthly Expense
-");
136         System.out.println("- Press 6 to Display an Expense");
137         System.out.println("- Press 7 to Quit\n \n ");
138         System.out.println("-----");
139         System.out.println("-----");
140         System.out.println("-----");
141     }
```

```
146 //Checking for an invalid input choice
147 if(sc.hasNextInt()) {
148     choice = sc.nextInt();
149
150     //This variable is sent as a argument to Update_RemoveRecord
151     //if its value is 0, it will remove a record
152     //if its value is 1, it will update a record.
153     int edit = 0;
154
155     //Creating a switch case branch based on the choice made by the
user
156     switch (choice) {
157         //First case adds the record to the user's expense table
158         case 1:
159             //System.out.println("CALLING AddExpense() ");
160             ExpenseCalclater.AddExpense() ;
161             break;
162
163         //Second case removes a record from the expense table
164         case 2:
165             //System.out.println("Calling RemoveExpense() ");
166             ExpenseCalclater.Update_RemoveExpense(edit);
167             break;
168
169         //Third case updates a record in the expense table
170         case 3:
171             // System.out.println("Calling UpdateAnExpense()");
172             edit=1;
173             ExpenseCalclater.Update_RemoveExpense(edit);
174             break;
175
176         //Fourth case displays the expense table
177         case 4:
178             // System.out.println("Calling DisplayTable()");
179             ExpenseCalclater.DisplayTable();
180             break;
181
182         //Fifth case calculates the total amount of expenses and
displays it
183         case 5:
184             //System.out.println("Calling CalculateExpense()");
185             ExpenseCalclater.CalculateExpense();
186             break;
187
188         //Sixth case outputs the following statement
189         case 6:
190             //System.out.println("Calling CompareExpenses() will be
implemented in release 2 as an extension of this project");
191             ExpenseCalclater.DisplayAnExpense();
192             break;
193
194         //Seventh case quits the program
195         case 7:
196             System.out.println("Exiting the EMS.\n\n");
197             System.exit(0);
```



```
198
199             //A default case which displays the message: invalid
choice made by the user
200             default:
201                 System.out.println("Invalid Choice!!! Please enter a Valid
Menu Choice.\n\n");
202             }
203         }
204
205         else {
206             System.out.println("Invalid Choice!!! Please enter a Valid
integer for Menu Choice. \n\n");
207         }
208     }
209 }
210
211 public static void main(String[] args) throws Exception{
212     try
213     {
214         //create instance of the CreateLoginForm
215         EMSLogin myuser = new EMSLogin();
216
217         //Wait to handle the data in the GUI thread to handle the
multithreading situation with Main Thread;
218         do{
219             //System.out.println("login inside do while" +loginsuccess);
220             System.out.println("
");
221             }
222             while(loginsuccess==0);
223
224             // System.out.println("login outside do while" +loginsuccess);
225             if(loginsuccess == 1){
226                 //Clear the console
227                 System.out.println("
");
228                 System.out.print('\u000C');
229                 System.out.println("\n WELCOME TO THE EXPENSE MANAGEMENT SYSTEM (EMS)
\n");
230                 myuser.DisplayUmenu();
231             }
232         }
233         catch(Exception e)
234         {
235             //handle exception
236             JOptionPane.showMessageDialog(null, e.getMessage());
237
238         }
239
240         //System.out.println("I am here in main\n\n");
241         //System.out.println("login " +loginsuccess);
242
243     }
244
245     /*
* Authentication: This method will be invoked when we click on the Login
246 button. It helps authenticate the data entered in the username and password
```

```
247      * Implementing an action event listener class with conditional statement
248      * JOptionPane is another component of Java GUI, it comes in handy when
displaying either a warning, success, or error message on the screen, as a dialog
box.
249      */
250      @Override
251      public void actionPerformed(ActionEvent e) {
252          String Username = username.getText();
253          String Password1 = Password.getText();
254
255          if (Username.equals("admin") && Password1.equals("admin123"))
256          {
257
258              JOptionPane.showMessageDialog(null, "Login Successful, You are
the " + Username.toUpperCase() + "!", "Login", JOptionPane.PLAIN_MESSAGE);
259              setVisible(false);
260              loginsuccess = 1;
261
262          }
263
264          else if (Username.equals("user") && Password1.equals("user123"))
265          {
266
267              JOptionPane.showMessageDialog(null, "Login
Successful!", "Login", JOptionPane.PLAIN_MESSAGE);
268              setVisible(false);
269              loginsuccess = 1;
270          }
271
272          else{
273              int result = JOptionPane.showConfirmDialog(null, "Username or
password mismatch. Please try again ", "Login
alert!", JOptionPane.YES_NO_OPTION, JOptionPane.QUESTION_MESSAGE);
274
275              if(result == JOptionPane.YES_OPTION){
276                  //Reset Username and Password field.
277                  username.setText("");
278                  Password.setText("");
279              }
280              else if (result == JOptionPane.NO_OPTION){
281                  setVisible(false);
282                  dispose();
283                  loginsuccess = 2;
284                  System.exit(0);
285              }
286              else if (result == JOptionPane.CLOSED_OPTION) {
287                  //System.out.println("Window closed without Selecting!");
288                  loginsuccess = 2;
289                  System.exit(0);
290              }
291
292          }
293      }
294 }
```